

SSD_Timer usage reference

1 Overview

The public version provides 3 Timer hardware timers, which can be configured as single trigger or cyclic trigger.

Timer is disabled by default, and customers can decide whether to enable or not.

2. TIMER control

2.1. Set Single/Cyclic Trigger

```
#define TIMER_ENABLE ( 0x1 ) / *Set cyclic trigger*/ #define  
TIMER_TRIG ( 0x2 ) / * Set single trigger*/  
  
// Choose one of two  
WRITE_WORD ( reg_base , TIMER_ENABLE | TIMER_INTERRUPT ); /*Set  
cyclic trigger*/  
WRITE_WORD ( reg_base , TIMER_TRIG | TIMER_INTERRUPT ); /*Set  
single trigger*/
```

2.2. Start the timer

```
//The following three timers are set to 2 seconds, and the
parameter unit is us
_set_timer(2000000, REG_BASE_TIMER0);
mdelay(50); //To start the timer at the same time, delay must be
added
_set_timer(2000000, REG_BASE_TIMER1);
mdelay(50); //To start the timer at the same time, you must add
delay
_set_timer(2000000, REG_BASE_TIMER2);
```

2.3. Turn off the timer

```
_reset_timer(REG_BASE_TIMER0);
_reset_timer(REG_BASE_TIMER1);
_reset_timer(REG_BASE_TIMER2);
```

2.4. Interrupt Handling

```
irqreturn_t interrupt_handle(int irq,void *dev_id)
{
    printk("====> hit %d \n", irq);
    return IRQ_HANDLED;
}
```

2.5. 完整demo

```
#include <linux/module.h>
#include <linux/random.h>
#include <linux/kthread.h>
#include <linux/delay.h>
#include <linux/delay.h>
#include <linux/interrupt.h>

#include <linux/irqdomain.h>    //struct irq_domain
#include <linux/of.h>           //of_find_compatible_node
#include <linux/irq.h>          //IRQ_TYPE_LEVEL_HIGH
```

```

#include <iqs.h>                                //INT_FIQ_TIMER_0

#define REG_BASE_TIMER0      (0xDE000000+0x1f000000+0x3020*2)
#define REG_BASE_TIMER1      (0xDE000000+0x1f000000+0x3040*2)
#define REG_BASE_TIMER2      (0xDE000000+0x1f000000+0x3060*2)
#define ADDR_TIMER_CTL        (0x0<<2)
#define TIMER_ENABLE          (0x1)  /*设置循环触发*/
#define TIMER_TRIG             (0x2)  /*设置单次触发*/
#define TIMER_INTERRUPT        (0x100) /*开启中断*/
#define ADDR_TIMER_HIT         (0x1<<2)
#define ADDR_TIMER_MAX_LOW     (0x2<<2)
#define ADDR_TIMER_MAX_HIGH    (0x3<<2)
#define ADDR_TIMER_DIV         (0x6<<2)

#define WRITE_WORD(_reg, _val)  {(*(volatile unsigned short*)
(_reg))) = (unsigned short)(_val); }

static int vIrqNum0=0;
static int vIrqNum1=0;
static int vIrqNum2=0;

irqreturn_t interrupt_handle(int irq,void *dev_id)
{
    printk("====> hit %d \n", irq);
    return IRQ_HANDLED;
}

void _set_timer(unsigned int usec, unsigned int reg_base)
{
    unsigned int interval;

    interval = usec*12;

    //set timer_clk to 12M for I2m/P3, (mcu_432m/12m)-1=0x23
    WRITE_WORD(reg_base+ ADDR_TIMER_DIV, 0x23);
    WRITE_WORD(reg_base+ ADDR_TIMER_MAX_LOW, (interval &0xffff));
    WRITE_WORD(reg_base+ ADDR_TIMER_MAX_HIGH, (interval >>16));
    WRITE_WORD(reg_base, 0);
    WRITE_WORD(reg_base, TIMER_ENABLE|TIMER_INTERRUPT); /*设置循环触
发*/
    //WRITE_WORD(reg_base, TIMER_TRIG|TIMER_INTERRUPT); /*设置单次触
发*/

}

void _reset_timer(unsigned int reg_base)
{

```

```

        WRITE_WORD(reg_base, 0);
    }

static __init int mdrv_init(void)
{
    printk(KERN_ALERT "Timer mdrv_init! \n");

    /*timer0*/
    {
        struct device_node *intr_node;
        struct irq_domain *intr_domain;
        struct irq_fwspec fwspec;

        intr_node = of_find_compatible_node(NULL, NULL,
"sstar,main-intc");
        intr_domain = irq_find_host(intr_node);
        if(!intr_domain)
            return -ENXIO;
        fwspec.param_count = 3; //refer to #interrupt-cells
        fwspec.param[0] = 0; //GIC_SPI
        fwspec.param[1] = INT_FIQ_TIMER_0; /* irqs.h */
        fwspec.param[2] = IRQ_TYPE_LEVEL_HIGH;
        fwspec.fwnode = of_node_to_fwnode(intr_node);
        vIrqNum0 = irq_create_fwspec_mapping(&fwspec);

        if (request_irq(vIrqNum0, interrupt_handle, 0, "timer0", (void
*)REG_BASE_TIMER0))
            return -EBUSY;
    }
#ifdef 1
    /*timer1*/
    {
        struct device_node *intr_node;
        struct irq_domain *intr_domain;
        struct irq_fwspec fwspec;

        intr_node = of_find_compatible_node(NULL, NULL,
"sstar,main-intc");
        intr_domain = irq_find_host(intr_node);
        if(!intr_domain)
            return -ENXIO;
        fwspec.param_count = 3; //refer to #interrupt-cells
        fwspec.param[0] = 0; //GIC_SPI
        fwspec.param[1] = INT_FIQ_TIMER_1; /* irqs.h */
        fwspec.param[2] = IRQ_TYPE_LEVEL_HIGH;
        fwspec.fwnode = of_node_to_fwnode(intr_node);
        vIrqNum1 = irq_create_fwspec_mapping(&fwspec);
    }
#endif
}

```

```

        if (request_irq(vIrqNum1, interrupt_handle, 0, "timer1", (void
*)REG_BASE_TIMER1))
            return -EBUSY;
    }
#endif
#if 2
    /*timer2*/
    {
        struct device_node *intr_node;
        struct irq_domain *intr_domain;
        struct irq_fwspec fwspec;

        intr_node = of_find_compatible_node(NULL, NULL,
"sstar,main-intc");
        intr_domain = irq_find_host(intr_node);
        if(!intr_domain)
            return -ENXIO;
        fwspec.param_count = 3; //refer to #interrupt-cells
        fwspec.param[0] = 0; //GIC_SPI
        fwspec.param[1] = INT_FIQ_TIMER_2; /* irqs.h */
        fwspec.param[2] = IRQ_TYPE_LEVEL_HIGH;
        fwspec.fwnode = of_node_to_fwnode(intr_node);
        vIrqNum2 = irq_create_fwspec_mapping(&fwspec);

        if (request_irq(vIrqNum2, interrupt_handle, 0, "timer2", (void
*)REG_BASE_TIMER2))
            return -EBUSY;
    }
#endif

    //以下3个timer定时设为2秒，参数单位为us
    _set_timer(2000000, REG_BASE_TIMER0);
    mdelay(50); //同时启动定时器，必须加delay
    _set_timer(2000000, REG_BASE_TIMER1);
    mdelay(50); //同时启动定时器，必须加delay
    _set_timer(2000000, REG_BASE_TIMER2);
    return 0;
}

static __exit void mdrv_exit(void)
{
    printk(KERN_ALERT " mdrv_exit!\n");

    _reset_timer(REG_BASE_TIMER0);
    _reset_timer(REG_BASE_TIMER1);
    _reset_timer(REG_BASE_TIMER2);
    free_irq (vIrqNum0, (void *)REG_BASE_TIMER0);

```

```

    free_irq (vIrqNum1, (void *)REG_BASE_TIMER1);
    free_irq (vIrqNum2, (void *)REG_BASE_TIMER2);

}

module_init(mdrv_init);
module_exit(mdrv_exit);

MODULE_AUTHOR("SSTAR");
MODULE_DESCRIPTION("ms cpu load driver");
MODULE_LICENSE("GPL");

```

2.6. Makefile

```

KERNEL_PATH = /home/work/kernel

MODULE_NAME = $(shell basename ${PWD})
obj-m := $(MODULE_NAME).o
$(MODULE_NAME)-y := mdrv_timer.o

EXTRA_CFLAGS      += -Idrivers/sstar/include
EXTRA_CFLAGS      += -Idrivers/sstar/include/infinity2m

all:
    make -C $(KERNEL_PATH) M=$(PWD) modules

clean:
    make -C $(KERNEL_PATH) M=$(PWD) clean

```